Estatly Guide: Team

The Estatly team feature explained file by file and line by line: how one component serves two routes, how an admin creates accounts from the browser, how workload counts are fetched in one request, and how every page stays fresh after a rename. With the decisions behind them, the changes made in review, interview questions and answers, and vocabulary.

Branch: refactor/team. Updated November 6, 2026. Matches the code in your repo after this review.

Contents

- 1. The big picture
- 2. Routes and pages
- 3. lib/validation.ts: the team schemas
- 4. api/apiTeam.ts
- 5. The data hooks
- 6. useTeamMemberList.ts
- 7. TeamMemberList.tsx
- 8. TeamMemberCard, Stat, MemberActions
- 9. The create form
- 10. TeamMemberEditForm.tsx
- 11. TeamSkeleton.tsx
- 12. Decisions and changes
- 13. Still open
- 14. Interview questions
- 15. Vocabulary
- 16. Self-test

1. The big picture

The team feature lists the people in the agency with their workload. Admins add people, edit their details, and deactivate or reactivate them. Managers can see the agents, read-only.

Who can do what

Role	/agents	/managers
Admin	See, add, edit, deactivate	See, add, edit, deactivate
Manager	See, read-only	Redirected away
Agent	Redirected away	Redirected away

How the data flows

Layer	File	Job
Route	App.tsx + RoleProtected	Who may open the page
Page	Agents.tsx, Managers.tsx	Pass the role
Component	TeamMemberList, TeamMemberCard, MemberActions, Stat, forms	Markup only
List hook	useTeamMemberList	Role check, data, edit state, active split
Data hooks	useTeam, useCreateTeamMember, useUpdateTeamMember, useSetMemberActive	Cache and refresh
API	apiTeam.ts	Validate, create accounts, count workload
Database	Trigger, RLS, get_user_role	New accounts start inactive; only admins update profiles

Deactivate, not delete

- People who leave keep their records: their listings, clients and viewings still point to them.
- Deactivated people lose all access (get_user_role returns nothing for them), and their card moves to a Deactivated section.
- Reactivating restores access in one tap. Nothing is lost, so the action needs no confirmation dialog.

2. Routes and pages

```
<Route path="/agents" element={
  <RoleProtected allowed={["admin", "manager"]}>
    <Agents />
  </RoleProtected>
} />
<Route path="/managers" element={
  <RoleProtected allowed={["admin"]}>
    <Managers />
  </RoleProtected>
} />
```

- RoleProtected redirects anyone whose role is not in the list. Managers may see agents; only admins see managers.
- This is user experience. RLS on user_profiles is the security: a manager calling the API directly still cannot change anyone.

```
// pages/Agents.tsx
export default function Agents() {
  return <TeamMemberList role="agent" />
}

// pages/Managers.tsx
export default function Managers() {
  return <TeamMemberList role="manager" />
}
```

- **Two routes, one component.** The pages only differ in the role they pass. The UI, the styling and the logic are shared.
- Each route gets its own URL, its own lazy-loaded chunk and its own RoleProtected rule, without duplicating any feature code.
- An interview line: "I parameterised the component instead of copying it."

3. lib/validation.ts: the team schemas

```
export const createTeamMemberSchema = z.object({
  full_name: z.string().trim()
    .min(2, "Full name is required")
    .max(80, "Name is too long"),
  email: z.string().trim()
    .min(1, "Email is required")
    .email("Enter a valid email"),
  password: z.string()
    .min(8, "At least 8 characters")
    // bcrypt only uses the first 72 bytes
    .max(72, "Password is too long"),
  role: z.enum(["manager", "agent"],
    { message: "Choose a role" }),
})

export type CreateTeamMemberValues =
  z.infer<typeof createTeamMemberSchema>
```

- **Renamed in this review** from createAgentSchema: it creates managers too.
- **One schema, one type.** z.infer derives the TypeScript type from the schema. Before, the same shape was also written by hand twice (CreateTeamInput and CreateAgentFormValues), and three copies can drift apart.
- role is an enum without admin: nobody can create an admin from this form.
- No .trim() on the password: spaces can be part of a password.

```
// Edit details (email and role are locked)
export const teamMemberFormSchema = z.object({
  full_name: ...,
  phone: z.string().trim()
    .max(20, "Phone number is too long")
    .regex(PHONE, "Use digits, spaces, +, -, or ()"),
  specialization: z.string().trim().max(80, "..."),
})

export const teamMemberSchema =
  teamMemberFormSchema.transform((values) => ({
    full_name: values.full_name,
    phone: values.phone || null,
    specialization: values.specialization || null,
  }))
```

- **Two schemas, two jobs.** The form schema works with strings for the inputs. The transform turns empty strings into null before saving, so the database never stores "".
- The edit schema has no email or role: those are locked after creation, so the form cannot send them.
- The profile page reuses these schemas (profileFormSchema = teamMemberFormSchema): the same fields with the same rules.

4. api/apiTeam.ts

createTeamMemberApi

```
export async function createTeamMemberApi(
  values: CreateTeamMemberValues
) {
  const { params } = await action({
    params: values,
    schema: createTeamMemberSchema,
  })
  const { full_name, email, password, role } = params
```

- **Added in this review:** every other API function validated through action(). This one trusted its input.
- The form already validates, but the API must not depend on the caller behaving. Same rules in both places, one schema.

```
  // 1. signUp switches the session to the new user,
  //    so keep the admin's
  const { data: sessionData } =
    await supabase.auth.getSession()
  const adminSession = sessionData.session
  if (!adminSession)
    throw new Error("You must be signed in to add a team member")
```

- **The problem:** supabase.auth.signUp in the browser signs you in as the account it creates.
- So the admin's tokens are saved first. getSession is fine here: we only need the tokens to restore, not proof of who the user is.

```
// 2. Create the account
//    (the database makes it an inactive agent)
const { data, error } = await supabase.auth.signUp({
  email,
  password,
  options: { data: { full_name } },
})
```

- options.data becomes user metadata. A database trigger reads full_name from it and creates the user_profiles row.
- The trigger always creates an **inactive agent**. Nobody can sign themselves up into a role or straight into the app.

```
  // 3. Switch back to the admin,
  //    whether signUp worked or not
  await supabase.auth.setSession({
    access_token: adminSession.access_token,
    refresh_token: adminSession.refresh_token,
  })

  if (error) throw toRequestError(error)
  if (!data.user) throw new Error("Account could not be created")
```

- **The order matters.** The session is restored before checking the error. If signUp failed and we threw first, the admin could be left in a broken session.
- Without this step, an admin who adds an agent would suddenly be signed in as that agent.

```
  // 4. As admin: give the role and activate
  const { error: activateError } = await supabase
    .from("user_profiles")
    .update({ role, is_active: true })
    .eq("user_id", data.user.id)

  if (activateError)
    throw toRequestError(activateError, "Team member")
}
```

- This runs as the admin again, so RLS allows it. Only an admin's update can set a real role and activate an account.
- **The rule lives in the database:** the browser only asks; the trigger and RLS decide.
- See Still open: this whole flow belongs in an Edge Function.

getTeamApi

```
const { data, error } = await supabase
  .from("user_profiles")
  .select(`*,
    listings:properties!properties_agent_id_user_profiles_fkey(count),
    clients:clients!clients_assigned_agent_id_user_profiles_fkey(count),
    viewings:viewings!viewings_agent_id_user_profiles_fkey(count)`)
  .eq("role", role)
```

- **Workload in one request.** Three counted joins: each person's listings, clients and viewings, counted by Postgres. No N+1 problem.
- The !fkey names say which foreign key links each table to user_profiles.
- Aliases (listings:, clients:, viewings:) give the counts readable names.

```
  // These filters apply to the counted rows,
  // not to the people
  .eq("clients.status", "active")
  .eq("viewings.status", "scheduled")
  .gte("viewings.scheduled_at", new Date().toISOString())
  .order("full_name")
```

- **A filter on a joined table** (clients.status) filters the rows being counted, not the parent rows. Every agent still appears; their count includes only active clients and upcoming scheduled viewings.
- This is the line to explain in an interview: it is the difference between "agents with active clients" and "agents, with a count of active clients".
- Sorted by name, because people look for a person by name.

```
const rows = (data ?? []) as unknown as (UserProfile & {
  listings: { count: number }[]
  clients: { count: number }[]
  viewings: { count: number }[]
})[]

return rows.map(({ listings, clients, viewings, ...profile }) => ({
  ...profile,
  listing_count: listings[0]?.count ?? 0,
  active_client_count: clients[0]?.count ?? 0,
  upcoming_viewing_count: viewings[0]?.count ?? 0,
}))
```

- The double cast exists because the Supabase client is untyped (same as clients).
- Destructuring with a rest property pulls the three join arrays out and keeps the profile.
- The new names say exactly what is counted: active_client_count, not clients. **Shaping at the boundary**: components get plain numbers.

updateTeamMemberApi and setTeamMemberActiveApi

```
const { data, error } = await supabase
  .from("user_profiles")
  .update({ is_active: isActive })
  .eq("user_id", userId)
  .select("id")

if (error) throw toRequestError(error, "Team member")
if (data.length === 0) {
  throw new ForbiddenError("Only admins can change access")
}
```

- **The silent RLS block again.** If a manager calls this directly, RLS changes 0 rows and returns no error. The length check turns that into a clear ForbiddenError.
- updateTeamMemberApi uses the same check after validating with teamMemberSchema.
- They filter by user_id (the auth id), not the profile's id, because that is what links everything else.

5. The data hooks

useTeam

```
export function useTeam(role: CreatableRole) {
  const { data: team, isLoading, error, refetch } = useQuery({
    queryKey: ["team", role],
    queryFn: () => getTeamApi(role),
  })
  return { team, isLoading, error, refetch }
}
```

- **role is in the key.** Agents and managers are cached separately. Everything a query depends on belongs in its key.
- queryFn is an arrow function so role is passed in when TanStack Query fetches.

useCreateTeamMember (renamed in this review)

```
export function useCreateTeamMember(role: CreatableRole) {
  const queryClient = useQueryClient()

  const { isPending, mutate: createMember } = useMutation({
    mutationFn: createTeamMemberApi,
    onSuccess: () => {
      toast.success(`${role === "agent" ? "Agent" : "Manager"} added`)
      queryClient.invalidateQueries({ queryKey: ["team", role] })
    },
    onError: (err) => toast.error(err.message),
  })

  return { isPending, createMember }
}
```

- Was useCreateTeam: it creates one member, not a team.
- Invalidates ["team", role] only: adding an agent cannot change the managers list.

useSetMemberActive

```
onSuccess: (_data, { isActive }) => {
  toast.success(isActive ? "Access restored" : "Access removed")
  queryClient.invalidateQueries({ queryKey: ["team"] })
},
```

- onSuccess receives the result and then the variables the mutation was called with. The result is unused, so it is named _data (an underscore means "intentionally unused").
- The toast is written from the variables: "Access restored" or "Access removed".

useUpdateTeamMember (the bug fix)

```
onSuccess: () => {
  toast.success("Details saved")
  queryClient.invalidateQueries({ queryKey: ["team"] })
  // These pages join agent:user_profiles(full_name, avatar)
  queryClient.invalidateQueries({ queryKey: ["clients"] })
  queryClient.invalidateQueries({ queryKey: ["properties"] })
  queryClient.invalidateQueries({ queryKey: ["property"] })
  queryClient.invalidateQueries({ queryKey: ["viewings"] })
  queryClient.invalidateQueries({ queryKey: ["dashboard"] })
},
```

- **The bug:** clients, properties, a single property, viewings and the dashboard all join the agent's name. Only ["team"] was invalidated, so a renamed agent kept their old name everywhere else.
- **Why six keys is cheap:** invalidateQueries only refetches queries that are on screen. The others are marked stale and refetch when visited.
- The same kind of bug as useEditClient in the clients branch. The rule: when you change a table, find every query that joins it.

Hook	Invalidates	Why
useCreateTeamMember	team, role	One list gains a card
useUpdateTeamMember	team, clients, properties, property, viewings, dashboard	Names and avatars are joined everywhere
useSetMemberActive	team	Only the card's section changes

6. useTeamMemberList.ts

New in this review. Same split as useClientsBoard: the hook owns logic and state, the component owns markup.

```
const { user } = useUser()
// Only admins manage people;
// managers see the agent team read-only
const isAdmin = user?.user_profile?.role === "admin"
```

- A derived value. It decides what to show; RLS decides what is allowed.

```
const { team, isLoading, error, refetch } = useTeam(role)
const { setActive } = useSetMemberActive()
const [editTarget, setEditTarget] = useState<TeamMember>()

const members = team ?? []
```

- editTarget: undefined means the edit sheet is closed; a member means it is open for them. One state instead of two that could disagree.
- members ?? [] lets the filters below run safely while loading.

```
return {
  team,
  // Deactivated people get their own section
  active: members.filter((m) => m.is_active),
  inactive: members.filter((m) => !m.is_active),
  isLoading, error, refetch, isAdmin, editTarget,
  openEdit: (member: TeamMember) => setEditTarget(member),
  closeEdit: () => setEditTarget(undefined),
  toggleActive: (member: TeamMember) =>
    setActive({
      userId: member.user_id,
      isActive: !member.is_active,
    }),
}
```

- **The split is in the hook, not the API:** it only decides how one page lays people out.
- team is still returned because DataRenderer needs the whole list for the loading, error and empty states.
- The hook returns intentions (openEdit, toggleActive), so the component never builds API arguments itself.

7. TeamMemberList.tsx

```
// The only differences between /agents and /managers
const COPY = {
  agent: { title: "Agents", empty: EMPTY_AGENTS },
  manager: { title: "Managers", empty: EMPTY_MANAGERS },
}
...
const { title, empty } = COPY[role]
```

- **A lookup object instead of if/else**: COPY[role] picks the right text. Adding a role means adding one line.
- Defined outside the component, so it is created once, not on every render.

```
function renderCards(members: TeamMember[]) {
  return (
    <ul className="grid grid-cols-1 gap-4
      sm:grid-cols-2 xl:grid-cols-3">
      {members.map((member) => (
        <li key={member.user_id}>
          <TeamMemberCard
            member={member}
            canManage={isAdmin}
            onEdit={() => openEdit(member)}
            onToggleActive={() => toggleActive(member)}
          />
        </li>
      ))}
    </ul>
  )
}
```

- **A render function, not a component.** It is called as renderCards(active), never as <RenderCards />. A component defined inside another component is a new type on every render, so React would unmount and remount its children each time.
- Used twice (active and deactivated), so the grid markup is written once.
- key={member.user_id}: stable and unique.

```
{isAdmin && (
  <CreateButton
    label={`Add ${role}`}
    title={`Add ${role === "agent" ? "an agent" : "a manager"}`}
    description="They sign in with the email and
      temporary password you set here."
  >
    {(onClose) => (
      <TeamMemberCreateForm role={role} onClose={onClose} />
    )}
  </CreateButton>
)}
```

- Only admins see the Add button. Managers see a read-only team.
- "an agent" vs "a manager": correct articles in the sheet title.
- children as a function (a render prop): CreateButton owns the open state and hands the form a way to close.

```
<DataRenderer
  data={team}
  ...
  loading={<TeamSkeleton />}
  empty={empty}
  render={() => (
    <div className="space-y-8">
      {active.length > 0 && renderCards(active)}
      {inactive.length > 0 && (
        <section className="space-y-3">
          <h2 className="text-sm font-medium
            text-muted-foreground">
            Deactivated · {inactive.length}
          </h2>
          {renderCards(inactive)}
        </section>
      )}
    </div>
  )}
/>
```

- active.length > 0 is a boolean, so a 0 can never be printed.
- Deactivated people are separated and labelled, not mixed in with the team.

```
<FormSheet open={Boolean(editTarget)}
  onOpenChange={(open) => !open && closeEdit()}
  title="Edit details">
  {editTarget && (
    <TeamMemberEditForm member={editTarget}
      onClose={closeEdit} />
  )}
</FormSheet>
```

- Mounting a fresh form per member: useForm reads defaultValues only once, when it mounts.

8. TeamMemberCard, Stat, MemberActions

Split in this review: MemberActions and Stat used to live inside the card file. Each is now its own component, so the card file is only the card.

TeamMemberCard

```
<article className={cn(
  "flex h-full flex-col rounded-xl border bg-card p-5",
  !member.is_active && "opacity-60"
)}>
```

- h-full: cards in the same grid row get the same height.
- Deactivated cards are dimmed, so their state is visible at a glance.
- cn merges classes and drops false values.

```
<UserAvatar name={member.full_name}
  src={member.avatar} size="lg" />
<div className="min-w-0 flex-1 pt-0.5">
  <h3 className="truncate font-medium">
    {member.full_name}
  </h3>
  <p className="truncate text-sm text-muted-foreground">
    {member.specialization || "No specialization"}
  </p>
</div>
{canManage && (
  <MemberActions member={member}
    onEdit={onEdit} onToggleActive={onToggleActive} />
)}
```

- || here, not ??: an empty specialization should also fall back.
- canManage hides the menu from managers.

```
<dl className="mt-5 grid grid-cols-3 divide-x
  rounded-lg bg-muted/60 py-3 text-center">
  <Stat label="Listings" value={member.listing_count} />
  <Stat label="Clients" value={member.active_client_count} />
  <Stat label="Viewings" value={member.upcoming_viewing_count} />
</dl>
```

- A description list of three label and value pairs: the person's workload.
- divide-x draws lines between the columns without extra elements.

```
<Button asChild variant="outline" size="sm" className="flex-1">
  <a href={`mailto:${member.email}`} title={member.email}>
    <Mail /> Email
  </a>
</Button>
```

- **asChild:** the Button passes its styles to the <a>, so the result is a real link that looks like a button. A link for navigation, a button for actions.
- title shows the full address on hover, since the button only says Email.

Stat

```
export default function Stat({ label, value }:
  { label: string; value: number }) {
  return (
    // flex-col-reverse: dt comes first in the HTML,
    // but the number shows on top
    <div className="flex flex-col-reverse px-2">
      <dt className="text-xs text-muted-foreground">{label}</dt>
      <dd className={cn("text-xl font-semibold tabular-nums",
        value === 0 && "text-muted-foreground")}>
        {value}
      </dd>
    </div>
  )
}
```

- **HTML order for meaning, CSS for looks.** dt must come before dd, so a screen reader reads "Listings, 4". flex-col-reverse puts the number on top visually.
- A zero is muted, so the real work stands out.

MemberActions

```
interface MemberActionsProps {
  member: TeamMember
  onEdit: () => void
  onToggleActive: () => void
}
```

- Its own props instead of Omit-ing the card's. The same fix as ClientActions.

```
{member.is_active ? (
  <DropdownMenuItem variant="destructive"
    onSelect={onToggleActive}>
    <UserX /> Deactivate
  </DropdownMenuItem>
) : (
  <DropdownMenuItem onSelect={onToggleActive}>
    <UserCheck /> Reactivate
  </DropdownMenuItem>
)}
```

- One callback, two labels: the menu shows the action that makes sense for the current state.
- Deactivate is red (destructive) because it removes access; Reactivate is not.
- aria-label on the trigger includes the member's name, like the clients cards.

9. The create form

useTeamMemberCreateForm.ts

```
const form = useForm<CreateTeamMemberValues>({
  resolver: zodResolver(createTeamMemberSchema),
  mode: "onTouched",
  // role is fixed by the page; there is no field for it
  defaultValues: { full_name: "", email: "", password: "", role },
})

const onSubmit = form.handleSubmit((values) => {
  createMember(values, { onSuccess: onClose })
})
```

- **Renamed in this review** from useTeamForm, so it says it is for creating.
- role comes from the page into defaultValues. It is validated like any other field, but the user never sees it.
- The sheet closes only on success, keeping the input on error.

TeamMemberCreateForm.tsx

```
<TextField control={control} name="email"
  id="member-new-email" label="Email" icon={Mail}
  type="email" placeholder="name@agency.com"
  autoComplete="off" required />
<PasswordField control={control} name="password"
  id="member-new-password"
  label="Temporary password"
  placeholder="At least 8 characters" required />
```

- **Fields inlined in this review.** They lived in TeamMemberFields, which had one user, needed its own props type (AgentFieldsProps) and was imported under another name (UserFields).
- **autoComplete="off":** the admin types someone else's details, so the browser should not fill in the admin's own email and password.
- "Temporary password" tells the admin what the field is for.

```
<p className="rounded-lg bg-muted p-3 text-sm">
  Share the email and password with the {role} yourself.
  They can change the password from their profile
  after signing in.
</p>
```

- The UI explains the process, because no invitation email is sent.

10. TeamMemberEditForm.tsx

```
const { control, handleSubmit } = useForm<TeamMemberFormValues>({
  resolver: zodResolver(teamMemberFormSchema),
  mode: "onTouched",
  defaultValues: {
    full_name: member.full_name ?? "",
    phone: member.phone ?? "",
    specialization: member.specialization ?? "",
  },
})
```

- null from the database becomes "" for the inputs; the schema's transform turns "" back into null on save.
- The form is small and used once, so it keeps useForm inline. Extract a hook if it grows.

```
const onSubmit = handleSubmit((values) => {
  updateMember(
    { userId: member.user_id, values },
    { onSuccess: onClose }
  )
})
```

- Sends user_id, the id the API filters by.

```
{/* Email is their sign-in,
    so it is shown but not editable */}
<div className="rounded-lg bg-muted p-3 text-sm">
  <p className="text-muted-foreground">Signs in with</p>
  <p className="truncate font-medium">{member.email}</p>
</div>
```

- Shows the email as information, not as a disabled input. A disabled input looks like a bug; a read-only box looks deliberate.
- Changing a sign-in email needs a verification flow, which this app does not have.

11. TeamSkeleton.tsx

```
<div role="status" className="grid grid-cols-1 gap-4
  sm:grid-cols-2 xl:grid-cols-3">
  <span className="sr-only">Loading team</span>
  {SKELETON_KEYS.slice(0, 6).map((key) => (
    <Skeleton key={key} className="h-56 rounded-xl" />
  ))}
</div>
```

- The same grid as the real cards, with cards of about the same height, so nothing shifts when the data arrives.
- role="status" with sr-only text for screen readers.

12. Decisions and changes

Use these as STAR stories: situation, task, action, result.

A rename that did not reach other pages

When an admin renamed an agent, the team page updated but clients, properties, viewings and the dashboard kept the old name, because they all join the agent's profile. I listed every query that joins user_profiles and invalidated them all. invalidateQueries only refetches what is on screen, so it costs almost nothing.

One shape written three times

The create form's shape existed as a Zod schema, a CreateTeamInput interface and a CreateAgentFormValues interface. I kept the schema, derived the type with z.infer, and deleted both interfaces. Now there is one source of truth.

An API that trusted its input

Every API function validated through my action() helper except the one that creates accounts. I added it, so the same schema protects the form and the API.

Removing an extra layer

The create form's fields lived in a separate component used only once, with its own props type and a renamed import. I inlined them. A split-out component should earn its place by being reused or hiding real complexity.

Splitting the card

The card file also contained the actions menu and the stat component. I moved each into its own file, so every file has one job and the card reads top to bottom.

Two routes, one component

Agents and managers need the same page with different data and permissions. Each route passes a role to one TeamMemberList, and a small COPY object holds the only text that differs.

13. Still open

Item	Why it matters
Account creation in the browser	signUp swaps the admin's session out and back. If setSession failed, the admin would be signed in as the new user; if step 4 failed, an inactive account would be left behind. It also needs email confirmation turned off. Fix: a Supabase Edge Function with the service role key and auth.admin.createUser, which checks the caller is an admin and never touches their session
as unknown as cast	Untyped Supabase client; generated database types would remove it
Edit sheet empties while closing	Same as clients: editTarget is cleared before the slide-out ends
Temporary password	The admin shares it by hand. An invitation email (auth.admin.inviteUserByEmail) would let people set their own

14. Interview questions

Q1. Why do /agents and /managers use the same component?

They show the same UI with different data and permissions. Each page passes a role to TeamMemberList, the query key includes the role, and a small COPY object holds the text that differs. No duplicated feature code.

Q2. How does an admin create an account in your app?

In the browser, signUp would sign the admin in as the new user, so I save the admin's session, call signUp, restore the session whether or not it worked, then update the new profile's role and activate it as the admin. A database trigger makes every new account an inactive agent, so only the admin's update gives real access.

Q3. What would you change about that?

Move it to a Supabase Edge Function using the service role key and auth.admin.createUser. The account is created on the server, the admin's session is never touched, and the function checks that the caller is an admin.

Q4. How do you get each person's workload without an N+1 problem?

Three counted joins in one select: listings, clients and viewings, each with (count). Postgres counts per person and returns the numbers with the rows.

Q5. What does .eq("clients.status", "active") do on a joined table?

It filters the joined rows being counted, not the people. Every agent still appears, and their client count includes only active clients.

Q6. You found a bug where renamed agents kept their old name. Explain it.

Clients, properties, viewings and the dashboard all join the agent's name, but editing a member only invalidated the team query. I invalidated every query that joins user_profiles. It is cheap, because only queries on screen refetch.

Q7. Why is role in the query key?

Agents and managers are different lists. Everything a query depends on belongs in its key, otherwise one page could show the other's cached data.

Q8. How is a manager stopped from editing agents?

The UI hides the menu and the Add button for non-admins. RLS on user_profiles only lets admins update, and the API checks for zero changed rows and throws a ForbiddenError. The UI is for experience, the database for security.

Q9. Why deactivate instead of delete?

People's listings, clients and viewings must keep pointing to them. Deactivation removes access but keeps the records, and it can be undone.

Q10. Why derive the type with z.infer?

The schema is the single source of truth. A hand-written interface beside it can drift. I had the same shape written three times and replaced two of them with z.infer.

Q11. Why is renderCards a function and not a component?

It is called as renderCards(list), so React sees plain JSX. A component defined inside another component becomes a new type on every render, and React would remount its children each time.

Q12. Why flex-col-reverse in Stat?

The HTML keeps the label before the value, the correct order for screen readers. CSS shows the number on top. Meaning in the HTML, looks in the CSS.

Q13. Why Button asChild around a link?

The element is a real link, because it navigates to mailto or tel, but it gets the button's styling. Links navigate; buttons do actions.

Q14. Why autoComplete="off" on the create form?

The admin types someone else's details. Without it, the browser might fill in the admin's own email and password.

Q15. Why show the email as text in the edit form instead of a disabled input?

It is the sign-in and cannot be changed here. A read-only box reads as deliberate; a disabled input looks broken.

Q16. When do you split a component into its own file?

When it has its own job, is reused, or makes the parent hard to read. I split MemberActions and Stat out of the card, but inlined TeamMemberFields, which had one user and only added a layer.

Q17. What does the underscore in (_data, { isActive }) mean?

The first argument is required by position but unused. The underscore says so on purpose, and linters ignore it.

Q18. What would you improve next?

Move account creation to an Edge Function, send invitation emails instead of temporary passwords, and type the Supabase client to remove the casts.

15. Vocabulary

Term	Meaning
Parameterised component	One component that changes behaviour through a prop, like role
Lookup object	An object used instead of if/else: COPY[role]
Session	The tokens that say who is signed in
Trigger	Database code that runs automatically on insert or update
Service role key	A server-only Supabase key that bypasses RLS
Edge Function	Server code hosted by Supabase
Counted join	A join that returns a count instead of rows: table(count)
Filter on a join	A condition on joined rows, not on the parent rows
z.infer	Derives a TypeScript type from a Zod schema
Single source of truth	Each fact defined in exactly one place
Render function	A function returning JSX, called directly, not as a component
asChild	Radix pattern: pass behaviour and styles to the child element
Soft deactivation	Removing access while keeping the records
Invalidation	Marking cached queries stale so they refetch

16. Self-test

Say each answer out loud in under 45 seconds, without looking.

1. Who can open /agents and /managers, and who can edit?
2. Why one component for two routes?
3. Walk me through creating a team member, step by step.
4. Why restore the session before checking the error?
5. What does the database trigger do for new accounts?
6. What would you change about account creation, and why?
7. How are the workload counts fetched?
8. What does a filter on a joined table do?
9. Explain the renamed-agent bug and its fix.
10. Why is invalidating six keys cheap?
11. Why z.infer instead of an interface?
12. Why deactivate instead of delete?
13. What does useTeamMemberList return, and why intentions?
14. Why is renderCards not a component?
15. Why flex-col-reverse in Stat?
16. Why did you inline TeamMemberFields but split MemberActions?
17. What would you improve next?